
Discussion / Q&A — prepared answers

Anticipated questions for Project 13 (VITS TTS, our droid fine-tune). Each has a **short answer** you can give cold, plus a deeper follow-up. Route: model/training → Dima, evaluation → Ilya, data/engineering → Emir.

The result & metrics

Q: Your WER got worse after fine-tuning (9% → 29%). Did it fail? No — that's the trade-off, not a failure. The goal was to change the *voice*, and **MCD dropped 10.5 → 6.2 dB**, meaning the output moved much closer to the droid timbre. Fine-tuning a voice into a model costs some intelligibility; we measured both sides honestly. Reducing the WER is our stated future work.

Q: Why is MCD the right metric here, not just WER? WER measures *intelligibility* (can you understand it). MCD measures *spectral closeness to the target voice* — exactly “did it learn the droid timbre.” For a voice-change project, MCD is the metric that shows the method worked; WER is the cost. We also report F0 RMSE (pitch) for completeness.

Q: What do you compare MCD/F0 against? The **B1 droid reference audio** from our dataset (the held-out RVC clips), not the original female LJSpeech — comparing a droid against a female voice would be meaningless. Lower MCD = closer to the droid we were targeting.

Q: Only 20 clips — is that enough? It's a held-out set (last 5%, unseen in training), enough to show the direction clearly and consistently. The numbers are small-sample, so we present the *trend* (MCD down, WER up) as the headline rather than over-claiming a precise value.

Q: Is Whisper circular / part of training? No. Whisper is an independent, frozen, off-the-shelf ASR. It never touches training — it's purely an intelligibility proxy on the output, which is exactly what the brief asks for.

The model

Q: Why VITS over a two-stage system (Tacotron2/FastSpeech2 + vocoder)? End-to-end: text → waveform in one model, no separate vocoder to train and keep in sync, no intermediate mel that gets predicted then re-synthesized. It also gives phoneme-level access and has a strong pretrained checkpoint.

Q: What is MAS? Monotonic Alignment Search — a dynamic-programming (Viterbi-like) search for the best **monotonic** map from text tokens to audio frames. Monotonic = left-to-right, no skipping or repeating, so it can't babble. It also yields the duration target for the duration loss. We implemented it from scratch.

Q: What are the five losses? Reconstruction (L1 on the mel, weight 45), KL (text-prior vs audio-posterior), duration, adversarial (LSGAN), and feature-matching. Reconstruction is weighted 45× because spectral accuracy matters most.

Q: Why a discriminator? What's MPD vs MSD? Trained on reconstruction alone the audio is blurry — L1 can't capture fine detail. The discriminator (HiFi-GAN style, written from scratch since HF ships VITS inference-only) fixes that. **MPD** folds the 1-D waveform by **prime periods (2, 7, 13, 29, 37, 73, 97, 113, 137)** to catch pitch/harmonics; **MSD** looks at 3 average-pooled time scales. Both expose feature maps → the feature-matching loss.

Q: What did you actually implement vs use from the library?

`transformers` ships VITS inference-only. We re-implemented the entire training stack: the training forward pass, MAS, the five losses, the discriminator (MPD+MSD), and the alternating GAN loop.

Training & debugging

Q: Report your training budget. Local **RTX 5060 (8 GB)**, **5 epochs**, **≈ 9.7 GPU-hours** for the final clean run (~1.9 h/epoch), plus ~3× more in failed runs and debugging. Batch 1 + gradient accumulation, AMP fp16, `num_workers=0` (Windows DataLoader). `lr 1e-4`, `segment 8192`, `grad-clip 10`, `disc-warmup 1500`.

Q: Why did early training fail, and how did you fix it? Three bugs. (1) The **duration loss was ~100× too large** — averaged wrong; fixing the normalization dropped it from ~300 to ~1.7. (2) **Gradient clipping was too tight**, killing the generator update; loosening it let reconstruction fall. (3) A **fresh discriminator corrupted the pretrained decoder**; we warmed up the discriminator alone for 1500 steps and lowered the LR. After that, reconstruction went 0.85 → 0.40 with correct rhythm.

Q: Your loss curves look converged — doesn't that prove it works? No, and that's the key lesson: a falling reconstruction loss says nothing about generation quality. Our broken runs had decreasing loss while generating garbage. Only the **inference-path WER** exposed the problems.

Q: Why `lr 1e-4` and not the from-scratch `2e-4`? `2e-4` is for training from scratch; for fine-tuning a pretrained model it's too aggressive and risks catastrophic forgetting. `1e-4` is the gentler fine-tune rate.

Data

Q: How did you build the dataset? We took LJSpeech (13,100 clips) and ran it through **RVC** (retrieval-based voice conversion) to repaint the timbre into a B1 droid, keeping the pronunciation — so the original transcripts stay valid. Published as `Dmitr13/ljspeech-b1` at 22050 Hz.

Q: Isn't training on VITS-then-RVC audio a kind of self-distillation? Partly, yes, and it's a fair critique. It's fine for our goal — *demonstrating voice adaptation*. For production you'd fine-tune on real target-voice recordings; the pipeline is identical, only the audio source changes.

Honest limitations

Q: Why doesn't it sound like a hard sci-fi robot? RVC transfers a *speaker's timbre*, not the ring-modulator “robot” buzz. So the result is a brighter, droid-ish voice rather than a mechanical one. That buzz lives in a separate DSP effect, not in the training data.

Q: Why do some prompts collapse (e.g. “Roger roger...” → 0.8 s)? On some short/punchy prompts the duration predictor collapses and crams the sentence into under a second of garbled audio. It's a known failure mode (shown on the failure slide).

Q: How would you cut the WER? Freeze the text/timing front-end (text encoder + duration predictor) during fine-tuning — for a timbre-only change the front-end shouldn't drift, and keeping it fixed prevents the duration collapse. That's our main next step, plus adding the Russian accent.

Q: Tokenizer — characters or phonemes? Phonemes — `vits-ljs` uses a phoneme tokenizer (espeak). Mispronunciations are grapheme-to-phoneme errors on rare words, not a missing-phoneme problem.

Engineering

Q: How did three people build this without colliding? Contract-first design: data, model, and evaluation talk only through small fixed interfaces (a manifest, an `EvalResult`), never each other's internals. That enabled parallel work and kept the model swappable. Plus an offline mock + smoke test to verify the wiring in seconds without a GPU.